

Guide de révision et maintenance rapide

Comprendre chaque application, suivre les flux métier, diagnostiquer les incidents et effectuer les opérations courantes du projet Django.

Projet : plateforme de réservation d'appartements à Sidi Rahal

Périmètre analysé : code présent dans le dépôt au 30 juillet 2026

Public : développeur, mainteneur, administrateur technique

Mode d'emploi du guide

Ce document est conçu à deux vitesses : les encadrés et tableaux suffisent pour une révision rapide ; les sections détaillées servent pendant une maintenance ou un diagnostic.

Lecture express en 10 minutes. Lire la fiche minute, l'architecture, le flux de réservation, les risques connus et la checklist d'intervention.

Sommaire

1. [Fiche minute et architecture générale](#)
2. [Application core — accueil public](#)
3. [Application accounts — utilisateurs et accès](#)
4. [Application apartments — catalogue et disponibilité](#)
5. [Application reservations — cycle de réservation](#)
6. [Application payments — paiement simulé et reçu](#)
7. [Application notifications — alertes internes](#)
8. [Application dashboard — pilotage administrateur](#)
9. [Services transverses, e-mails et PDF](#)
10. [Modèle de données et règles essentielles](#)
11. [Installation, configuration et démarrage](#)
12. [Maintenance courante, sauvegardes et déploiement](#)
13. [Diagnostic rapide et points de vigilance](#)
14. [Checklists et aide-mémoire](#)

Convention. « Application » désigne ici un module Django. Le projet contient sept applications internes, deux services métier et le paquet de configuration webbap.

1. Fiche minute et architecture générale

Résumé technique

Élément	Valeur
Framework	Django 6.0.7, Python, templates Django rendus côté serveur
Base de données	PostgreSQL si <code>DB_NAME</code> existe ; sinon SQLite dans <code>db.sqlite3</code>
Authentification	E-mail/mot de passe et Google OAuth via <code>django-allauth</code>
Fichiers	Statique dans <code>static/</code> , médias envoyés dans <code>media/</code>
E-mails	SMTP si <code>EMAIL_HOST</code> est défini, sinon sortie console
Paielement	Simulation locale par carte ; aucune transaction bancaire externe
Rapports	Export Excel avec <code>openpyxl</code> ; confirmation PDF avec Pillow
Langue / fuseau	Français, <code>Africa/Casablanca</code> , dates stockées avec gestion de fuseau

Carte des responsabilités

Module	Responsabilité	Données principales
<code>core</code>	Page d'accueil publique	Aucune
<code>accounts</code>	Compte, rôles, connexion, profil, Google OAuth	Utilisateur
<code>apartments</code>	Catalogue, photos, vacances, recherche de disponibilité	Appartement, Photo, <code>PeriodeVacances</code>
<code>reservations</code>	Création, politiques, statuts, annulation, expiration	Reservation
<code>payments</code>	Saisie factice, validation, confirmation, reçu	Paielement
<code>notifications</code>	Notifications internes et compteur de non-lues	Notification
<code>dashboard</code>	Indicateurs et export Excel administrateur	Aucune table propre
<code>services</code>	Orchestration métier et génération de confirmation	Aucune table propre
<code>webbap</code>	Configuration, routage racine, WSGI/ASGI	Paramètres globaux

Architecture d'une requête

Navigateur → `webbap/urls.py` → `application/urls.py` → vue → formulaire / service /

modèle → base → template HTML

Les vues contrôlent l'accès et l'affichage. Les formulaires valident les entrées. Les services portent les opérations qui touchent plusieurs modèles. Les modèles conservent les invariants importants, par exemple l'expiration ou la confirmation d'une réservation.

Rôles applicatifs

Rôle	Accès	Règle spéciale
Client régulier	Catalogue, réservations, paiement, profil, notifications	Bloqué sur les appartements concernés pendant une période de vacances
Enseignant	Mêmes fonctions client	Peut réserver pendant les vacances et bénéficie actuellement de 15 % de réduction
Administrateur	Tableau de bord, gestion, administration Django	Ne peut pas créer de réservation client

À retenir immédiatement. Le champ `role` et les drapeaux Django `is_staff/is_superuser` sont distincts. L'accès au métier administrateur dépend de `est_administrateur()` ; l'accès à `/admin/` dépend aussi de `is_staff`.

2. Application `core` — accueil public

`core` est la porte d'entrée la plus simple : elle rend la page d'accueil sans base de données ni authentification.

Route	/
Vue	<code>core.views.accueil</code>
Template	<code>core/templates/core/accueil.html</code>
Dépendances	Template global, CSS/JavaScript et images statiques

Fonctionnement

Une requête GET sur la racine appelle une vue qui rend directement le template. Les liens conduisent vers la recherche d'appartements, les comptes et les sections de localisation ou de services.

Maintenance rapide

- Contenu éditorial et structure : modifier `core/templates/core/accueil.html`.
- Style partagé : modifier `static/css/site.css`.
- Comportement du menu et des messages : modifier `static/js/site.js`.
- Après modification statique en production : exécuter `collectstatic` et invalider le cache si nécessaire.

Piège fréquent. Changer une image dans `media/` ne remplace pas une image référencée dans `static/`. Les médias sont des contenus envoyés par l'application ; les fichiers statiques font partie du code livré.

3. Application `accounts` — utilisateurs et accès

Cette application remplace l'utilisateur Django standard par un utilisateur identifié par son e-mail. Elle centralise les rôles, l'inscription, la connexion, le profil et l'adaptation des comptes Google.

Modèle `utilisateur`

Champ / méthode	Rôle
<code>email</code>	Identifiant unique de connexion ; aucun nom d'utilisateur classique
<code>nom</code> , <code>prenom</code> , <code>telephone</code>	Identité et contact
<code>role</code>	<code>CLIENT_REGULIER</code> , <code>ENSEIGNANT</code> OU <code>ADMINISTRATEUR</code>
<code>matricule</code>	Obligatoire dans le formulaire lorsqu'un enseignant s'inscrit
<code>taux_reduction()</code>	Retourne 15 % pour un enseignant, 0 % sinon
<code>is_active</code>	Permet de désactiver un compte sans le supprimer
<code>is_staff</code>	Autorise l'accès à l'administration Django

Flux de connexion

Inscription ou Google → création de `utilisateur` → session Django → redirection selon

le rôle

- Un client ou enseignant connecté arrive sur « Mes réservations ».
- Un administrateur arrive sur le tableau de bord.
- Le paramètre `next` est accepté uniquement s'il pointe vers l'hôte courant.
- La déconnexion exige une requête POST, ce qui limite les déconnexions déclenchées par un lien externe.

Routes utiles

URL	Fonction	Accès
<code>/accounts/inscription/</code>	Créer un compte client ou enseignant	Public
<code>/accounts/connexion/</code>	Se connecter par e-mail	Public
<code>/accounts/google/login/</code>	Démarrer Google OAuth via allauth	Public si configuré

URL	Fonction	Accès
/accounts/profil/	Voir le profil	Connecté
/accounts/profil/modifier/	Modifier identité et contact	Connecté
/accounts/mot-de-passe-oublie/	Envoyer un lien de réinitialisation	Public

Google OAuth

Le bouton Google n'est actif que si `GOOGLE_OAUTH_CLIENT_ID` et `GOOGLE_OAUTH_CLIENT_SECRET` sont présents. L'adaptateur copie prénom, nom et e-mail ; un nouvel utilisateur Google devient client régulier. Le callback local attendu est :

```
http://localhost:8000/accounts/google/login/callback/
```

Points de maintenance

- Modifier les rôles ou la réduction dans `accounts/models.py`, puis adapter formulaires, tests et affichage.
- Ne jamais proposer le rôle administrateur dans le formulaire public.
- Préférer `is_active=False` à la suppression d'un client ayant des réservations protégées.
- Tester l'envoi SMTP pour le mot de passe oublié ; avec le backend console, le lien apparaît seulement dans le terminal.
- Mettre à jour les URI OAuth à chaque nouveau domaine ou passage en HTTPS.

4. Application `apartments` — catalogue et disponibilité

Elle gère les logements, leurs photos, les périodes réservées aux enseignants et la recherche des logements libres pour un intervalle de dates.

Modèles

Modèle	Champs essentiels	Règle
Appartement	Titre, description, prix/nuit, capacité, disponible	Prix positif, capacité ≥ 1
Photo	Image, légende, indicateur principale	Fichier placé sous <code>media/apartements/</code>
PériodeVacances	Appartement, début, fin, libellé	Fin strictement postérieure au début

Recherche de disponibilité

Les intervalles utilisent la convention semi-ouverte : arrivée incluse, départ exclu. Une réservation du 10 au 12 occupe donc les nuits du 10 et du 11 ; une nouvelle arrivée le 12 est autorisée.

Dates valides → appartements actifs – réservations confirmées – attentes de moins de 24

h – vacances interdites au profil

Une réservation annulée, rejetée, terminée ou expirée ne bloque pas. Une attente vieille de plus de 24 h ne bloque plus, même avant la mise à jour explicite de son statut.

Prix

```
montant = prix_par_nuit × nombre_de_nuits × (1 - taux_de_reduction)
```

Le résultat est arrondi à deux décimales. Actuellement, seul l'enseignant reçoit une réduction de 15 %. Cette remise s'applique à toutes ses réservations, pas uniquement aux vacances.

Routes

URL	Fonction	Accès
<code>/appartements/</code>	Recherche paginée, 12 résultats/page	Public

URL	Fonction	Accès
/appartements/<id>/	Détail, photos et vacances	Public
/appartements/gestion/liste/	Liste de gestion	Administrateur
/appartements/gestion/creer/	Créer et joindre éventuellement une photo	Administrateur
/appartements/gestion/<id>/modifier/	Modifier et ajouter éventuellement une photo	Administrateur
/appartements/gestion/vacances/	Gérer les périodes	Administrateur

Maintenance rapide

- Mettre `disponible=False` pour retirer temporairement un logement sans supprimer son historique.
- La suppression d'un appartement ayant des réservations est bloquée par `PROTECT`.
- L'interface métier ajoute une photo à la fois ; l'administration Django permet une gestion plus complète.
- Il n'existe pas de règle garantissant une seule photo « principale » : plusieurs peuvent être marquées.
- Après déplacement de `MEDIA_ROOT`, vérifier les permissions d'écriture et le service des médias en production.

5. Application `reservations` — cycle de réservation

C'est le cœur du métier. Elle crée une réservation de manière atomique, attribue un numéro, bloque temporairement les dates, impose l'acceptation des politiques et contrôle l'annulation.

Cycle des statuts

EN_ATTENTE → paiement → CONFIRMEE | → 24 h → EXPIREE | → annulation → ANNULEE

Les statuts `TERMINEE` et `REJETEE` existent dans le modèle, mais aucun automatisme du code analysé ne les attribue. Ils peuvent être positionnés depuis l'administration ou par une future logique métier.

Création détaillée

1. L'utilisateur doit être authentifié et ne pas être administrateur.
2. L'appartement est relu avec verrou transactionnel.
3. La disponibilité est recalculée pour éviter deux réservations concurrentes.
4. Une période de vacances bloque le client régulier, mais pas l'enseignant.
5. Le montant est calculé et figé dans `montant_total`.
6. Un numéro aléatoire unique au format `aaAA000000` est créé.
7. La réservation démarre en attente et un e-mail de politiques est envoyé.

Transaction. Le verrouillage se trouve dans `services/reservation_services.py`. PostgreSQL est préférable en production pour bénéficier d'un vrai verrouillage par ligne ; SQLite a un comportement de verrouillage plus global.

Fenêtre de paiement de 24 heures

`date_expiration = date_reservation + 24 h`. Après cette limite, une réservation en attente est marquée expirée quand un écran ou une opération appelle la logique d'expiration. Les recherches de disponibilité ignorent directement les attentes trop anciennes.

Important. Il n'existe pas de tâche planifiée qui expire toutes les réservations exactement à l'échéance. L'état est corrigé lors de l'ouverture de certaines pages ou opérations. Pour un statut à jour en permanence, ajouter une commande de gestion exécutée périodiquement.

Annulation

- Une réservation confirmée ne peut pas être annulée.

- La présence d'un paiement payé interdit également l'annulation.
- Une annulation valide crée une notification interne pour le client.
- Les conditions affichées indiquent « non annulable, non remboursable » après paiement.

Routes principales

URL	Fonction	Accès
/reservations/appartement/<id>/reserver/	Créer une réservation	Client connecté
/reservations/mes-reservations/	Historique du client	Connecté
/reservations/<id>/	Détail et récapitulatif	Propriétaire ou administrateur
/reservations/<id>/politiques/	Lire puis accepter les politiques	Propriétaire
/reservations/<id>/annuler/	Annuler avant paiement	Propriétaire, POST
/reservations/gestion/liste/	Filtrer et gérer	Administrateur

Maintenance rapide

- Ne jamais modifier les dates ou le montant d'une réservation payée sans procédure métier explicite.
- Pour rechercher une réservation, utiliser son numéro ou l'e-mail client dans /admin/.
- Une suppression administrateur efface aussi son paiement associé par cascade.
- En cas d'échec SMTP à la création, la réservation reste créée et l'utilisateur voit un avertissement.
- Après modification d'une règle, ajouter un test dans reservations/tests.py : ce fichier couvre la majorité des scénarios critiques.

6. Application `payments` — paiement simulé et reçu

L'application simule un paiement par carte, confirme la réservation puis envoie une confirmation et un PDF. Elle ne communique avec aucune banque et ne stocke pas les champs de carte.

Ce n'est pas un paiement réel. Le formulaire vérifie seulement le format : 16 chiffres, expiration au format MM/AA et CVV de 3 ou 4 chiffres. Il ne vérifie ni Luhn, ni la date d'expiration, ni l'autorisation bancaire. Ne pas présenter cette version comme un système d'encaissement en production.

Modèle `Paie`ment

Élément	Comportement
Relation	Un seul paiement par réservation grâce à <code>OneToOneField</code>
Méthode	Uniquement <code>CARTE</code>
Statuts	<code>EN_ATTENTE</code> , <code>PAYE</code> , <code>ECHEC</code>
Montant	Doit correspondre exactement au montant figé de la réservation
Effectuer	Verrouille la réservation, revalide toutes les conditions, marque payé et confirme

Flux

Politiques acceptées → formulaire factice → `Paie`ment.`PAYE` → `Reservation.CONFIRMEE` →

e-mail + PDF → reçu web

La confirmation base de données est terminée avant l'envoi de l'e-mail. Un échec SMTP n'annule donc pas le paiement ; un message avertit le client que son reçu reste disponible.

Routes

URL	Fonction	Accès
<code>/paiements/reservation/<reservation_id>/</code>	Payer	Propriétaire, réservation admissible
<code>/paiements/recu/<paiement_id>/</code>	Voir le reçu	Propriétaire, paiement payé
<code>/paiements/gestion/</code>	Filtrer les paiements	Administrateur

Passage à une vraie passerelle

1. Créer l'intention de paiement côté serveur auprès du prestataire.
2. Ne jamais faire transiter le numéro de carte brut par Django ; utiliser le composant sécurisé du prestataire.
3. Confirmer uniquement depuis un webhook signé et idempotent.
4. Ajouter un identifiant externe, les erreurs, remboursements et journaux d'audit.
5. Repenser l'annulation, la politique de remboursement et les tests de concurrence.

7. Application `notifications` — alertes internes

Elle fournit une boîte de notifications interne simple. Une notification appartient à un client, possède une date d'envoi et peut être marquée comme lue.

Fonctionnement

- `envoyer()` renseigne `date_envoi` une seule fois.
- `marquer_lue()` rend l'opération idempotente.
- Un context processor compte les notifications non lues à chaque page authentifiée.
- Le marquage comme lu est limité au propriétaire et exige POST.

URL	Fonction
<code>/notifications/</code>	Liste complète du compte connecté
<code>/notifications/<id>/lue/</code>	Marquer une notification comme lue

Limites actuelles

- Le code métier analysé crée une notification lors d'une annulation, pas lors d'une confirmation.
- Aucun e-mail ou push n'est envoyé par cette application ; les e-mails de réservation sont séparés.
- Le compteur effectue une requête sur chaque page connectée. À grande échelle, prévoir cache ou annotation.
- Il n'existe pas d'action « tout marquer comme lu » ni de purge automatique.

8. Application `dashboard` — pilotage administrateur

Le tableau de bord agrège les réservations et paiements sur une période. Sans dates fournies, il couvre les 30 derniers jours jusqu'au lendemain de la date courante.

Indicateurs

Indicateur	Calcul
Taux d'occupation	Jours occupés par réservations confirmées/terminées ÷ appartements disponibles ÷ jours de période
Revenu total	Somme des paiements payés dont la date de paiement appartient à la période
Compteurs	Total, confirmées et annulées parmi les séjours chevauchant la période
Réservations par mois	Regroupement selon le mois de la date d'arrivée

Interprétation. Le revenu est filtré par date de paiement, alors que les compteurs sont filtrés par chevauchement des dates de séjour. Deux chiffres de la même période ne décrivent donc pas forcément le même ensemble de réservations.

Routes

`/tableau-de-bord/` Indicateurs administrateur

`/tableau-de-bord/export/` Fichier Excel des réservations de la période

Maintenance rapide

- Modifier les définitions d'indicateurs dans `dashboard/views.py` et documenter la nouvelle formule.
- Le taux d'occupation additionne les jours ; il peut dépasser 100 % si les données contiennent des chevauchements anormaux.
- L'export utilise `openpyxl` et conserve les dates comme cellules date.
- Pour de gros volumes, remplacer la boucle de calcul des jours occupés par un calcul SQL ou une table d'agrégats.

9. Services transverses, e-mails et PDF

`services/reservation_services.py`

Ce module est le point d'entrée métier recommandé pour créer une réservation et chercher des appartements. Il évite de disperser les règles de disponibilité, de priorité enseignant et de concurrence dans les vues.

Fonction	Responsabilité
<code>creer_reservation(...)</code>	Transaction, verrou, autorisation, disponibilité, vacances, prix, validation et création
<code>chercher_appartements_disponibles(...)</code>	Requête optimisée avec sous-requêtes d'existence pour les conflits

`services/confirmation_reservation.py`

Après paiement, ce service prépare le contexte, dessine deux pages A4 avec Pillow, les assemble en PDF puis joint le document à un e-mail texte + HTML.

- Page 1 : numéro, dates, hébergement, client, tarif et montant.
- Page 2 : non-remboursement, garantie, attribution, départ tardif, non-présentation et coordonnées.
- Polices recherchées : Noto Sans du projet/système, puis DejaVu Sans, puis police par défaut.
- Nom de pièce jointe : `confirmation-reservation-<numero>.pdf`.

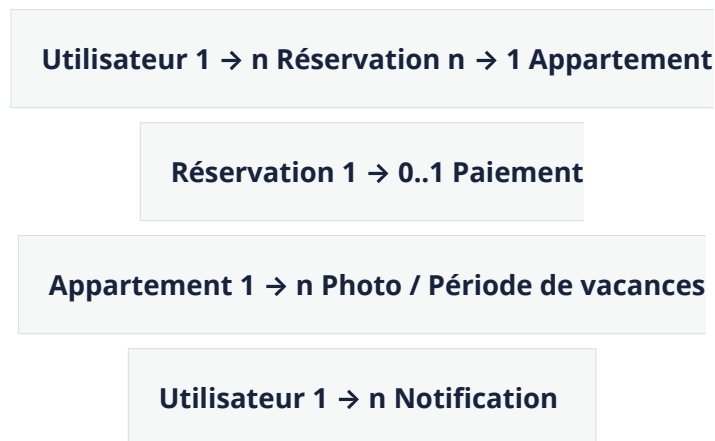
E-mails envoyés

Événement	Contenu	Échec
Réservation créée	Politiques et lien d'acceptation	Réservation conservée, avertissement écran
Paiement confirmé	Confirmation texte/HTML + PDF	Paiement conservé, reçu web disponible
Mot de passe oublié	Lien de réinitialisation Django	Dépend du backend e-mail configuré

Réessai manuel. Une confirmation peut être renvoyée depuis le shell Django avec prudence : charger un paiement payé, puis appeler `envoyer_email_confirmation(paiement)`. Vérifier d'abord l'adresse du destinataire pour éviter un envoi erroné.

10. Modèle de données et règles essentielles

Relations



Comportement lors d'une suppression

Suppression demandée	Effet
Utilisateur ayant des réservations	Bloquée : la réservation protège le client
Appartement ayant des réservations	Bloquée : la réservation protège l'appartement
Appartement sans réservation	Supprime aussi ses photos et périodes de vacances
Réservation	Supprime aussi son paiement éventuel
Utilisateur sans réservation	Supprime aussi ses notifications

Invariants à ne pas casser

- Date de fin strictement postérieure à la date de début.
- Une seule réservation bloquante par appartement et intervalle.
- Un administrateur ne réserve pas.
- Un client régulier ne réserve pas pendant une période de vacances concernée.
- Le paiement correspond au montant figé et exige les politiques acceptées.
- Une réservation payée est confirmée et non annulable.
- Numéro de réservation unique au format imposé.

Validation à deux niveaux. Certaines règles existent dans les formulaires et modèles, mais la non-superposition des réservations est une règle applicative, pas une contrainte d'exclusion en base. Tout nouveau chemin de création doit appeler le service métier.

11. Installation, configuration et démarrage

Démarrage local

```
cd /chemin/vers/appartements

python -m venv venv

source venv/bin/activate

pip install -r requirements.txt

cp .env.example .env

python manage.py migrate

python manage.py createsuperuser

python manage.py runserver
```

Application : <http://127.0.0.1:8000/> — Administration : <http://127.0.0.1:8000/admin/>.

Variables d'environnement

Variable	Usage	Conseil production
SECRET_KEY	Signature cryptographique Django	Longue, aléatoire, secrète ; aucun défaut
DEBUG	Mode de diagnostic	False
ALLOWED_HOSTS	Hôtes HTTP autorisés, séparés par virgules	Domaines exacts
DB_NAME/USER/PASSWORD/HOST/PORT	Active et configure PostgreSQL	Compte dédié, secret hors dépôt
GOOGLE_OAUTH_CLIENT_ID/SECRET	Connexion Google	Callback HTTPS enregistré
EMAIL_HOST/PORT/USER/PASSWORD	SMTP	Mot de passe d'application ou fournisseur transactionnel
EMAIL_USE_TLS	Chiffrement SMTP	True avec le port adapté
DEFAULT_FROM_EMAIL	Expéditeur par défaut	Domaine autorisé par le fournisseur
FOSE_SAFAR_*	Marque, adresse, contact,	Vérifier avant toute confirmation client

Variable	Usage	Conseil production
	horaires, garantie	

Valeurs par défaut permissives. Sans variables, le code utilise une clé de développement et `DEBUG=True`. Une production doit fournir explicitement les secrets et paramètres de sécurité.

Choix de base

La seule présence d'une valeur non vide dans `DB_NAME` active PostgreSQL. Sinon, Django utilise SQLite. Un serveur PostgreSQL arrêté provoque une erreur de connexion pour les commandes qui accèdent aux données, même si `db.sqlite3` existe.

Commandes de contrôle

```
python manage.py check

python manage.py showmigrations

python manage.py makemigrations --check --dry-run

python manage.py test

python manage.py collectstatic --noinput
```

État observé pendant la rédaction. `manage.py check` ne signale aucun problème et les 27 tests passent avec SQLite. L'environnement courant pointe vers un PostgreSQL local non joignable. La base SQLite présente aussi trois migrations internes non appliquées ; exécuter `migrate` sur la base réellement utilisée.

12. Maintenance courante, sauvegardes et déploiement

Avant chaque intervention

- Identifier la base réellement ciblée par les variables d'environnement.
- Faire une sauvegarde avant migration ou modification de données.
- Vérifier `git status` et préserver les changements existants.
- Reproduire l'incident et noter l'URL, le rôle, le numéro de réservation et l'heure.
- Tester sur une copie ou un environnement de préproduction.

Cycle d'une modification de modèle

```
python manage.py makemigrations

python manage.py sqlmigrate nom_application 000X

python manage.py migrate

python manage.py test
```

Versionner la migration avec le code. Ne pas modifier une ancienne migration déjà exécutée en production sauf stratégie maîtrisée. Pour une colonne obligatoire sur une table remplie, préparer une migration de données ou une transition en plusieurs étapes.

Sauvegarde SQLite

```
# Application arrêtée :

cp db.sqlite3 sauvegardes/db-AAAA-MM-JJ.sqlite3


# Ou avec l'outil SQLite :

sqlite3 db.sqlite3 ".backup sauvegardes/db-AAAA-MM-JJ.sqlite3"
```

Sauvegarde PostgreSQL

```
pg_dump --format=custom --file=fose-safar-AAAA-MM-JJ.dump NOM_BASE

# Restauration dans une base vide :

pg_restore --clean --if-exists --dbname=NOM_BASE_CIBLE fose-safar-AAAA-MM-JJ.dump
```

Toujours tester une restauration. Une sauvegarde non restaurée au moins une fois n'est pas une garantie de reprise.

Fichiers médias

La sauvegarde de base ne contient pas les images. Sauvegarder séparément `media/` et conserver sa cohérence avec la base. En production, Django ne sert les médias que lorsque `DEBUG=True` dans cette configuration : le serveur web ou un stockage objet doit les servir.

Checklist de déploiement

- ❑ `DEBUG=False`, clé secrète forte, hôtes exacts et HTTPS.
- ❑ PostgreSQL disponible, sauvegardé et migrations appliquées.
- ❑ `collectstatic` exécuté ; statiques et médias servis correctement.
- ❑ SMTP testé avec une adresse de test ; expéditeur authentifié.
- ❑ Origines et callbacks Google OAuth en HTTPS configurés.
- ❑ Variables `FOSE_SAFAR_*` relues dans le PDF de confirmation.
- ❑ Tests exécutés, puis test manuel inscription → réservation → paiement → e-mail.
- ❑ Journalisation, supervision, alertes et procédure de retour arrière prévues.

13. Diagnostic rapide et points de vigilance

Matrice d'incidents

Symptôme	Causes probables	Contrôle / action
Erreur de connexion à la base	DB_NAME active PostgreSQL, serveur arrêté, hôte/port ou identifiants erronés	Relire l'environnement ; tester PostgreSQL ; ne pas supposer que SQLite est utilisée
« no such column/table »	Migrations non appliquées ou mauvaise base	<code>showmigrations</code> , puis sauvegarde et <code>migrate</code>
Le bouton Google est désactivé	Client ID ou secret absent	Vérifier les deux variables et redémarrer le processus
Erreur OAuth <code>redirect_uri_mismatch</code>	Callback exact non déclaré chez Google	Ajouter protocole, domaine, port et chemin exacts dans Google Cloud
Aucun e-mail reçu	Backend console, SMTP refusé, mot de passe d'application, spam	Contrôler variables, terminal et journaux ; envoyer un e-mail depuis le shell
Appartement absent des résultats	Inactif, réservation bloquante, attente récente, vacances et profil régulier	Contrôler les dates semi-ouvertes, statuts, date de création et rôle
Paieement inaccessible	Réservation expirée/non en attente ou politiques non acceptées	Ouvrir le détail, vérifier statut, échéance et <code>politiques_acceptees_le</code>
Paieement confirmé mais pas d'e-mail	SMTP/PDF a échoué après la transaction	Le reçu reste valide ; corriger SMTP puis renvoyer la confirmation avec contrôle du destinataire
Image absente en production	Média non copié/servi, permissions, URL	Vérifier fichier sous <code>MEDIA_ROOT</code> , serveur média et droits
403 sur une page de gestion	Utilisateur authentifié sans rôle administrateur	Contrôler <code>role</code> ; pour <code>/admin/</code> , contrôler aussi <code>is_staff</code>
Statut en attente après 24 h	Expiration paresseuse, aucune page déclenchante visitée	Ouvrir liste/détail ou mettre en place une commande planifiée

Risques techniques prioritaires

1. **Paiement simulé** : remplacer avant tout encaissement réel.
2. **Paramètres de développement par défaut** : rendre les secrets obligatoires en production.
3. **Expiration non planifiée** : ajouter une tâche périodique si les statuts doivent être exacts sans consultation.
4. **Journalisation minimale** : configurer logs structurés, suivi des erreurs et corrélation par numéro de réservation.
5. **Tests concentrés** : compléter les tests dédiés à payments, notifications et dashboard, actuellement presque vides.
6. **Pas de statut terminé automatique** : décider quand un séjour devient `TERMINEE`.
7. **Médias locaux** : prévoir stockage durable et sauvegarde en production.

Requêtes utiles dans le shell Django

```
python manage.py shell

from reservations.models import Reservation

from payments.models import Paiement

Reservation.objects.filter(statut="EN_ATTENTE").count()

Reservation.expirer_en_attente()

Paiement.objects.filter(statut="PAYE").count()
```

Prudence. `Reservation.expirer_en_attente()` modifie les données. L'exécuter uniquement sur la base identifiée, après vérification de l'heure et du fuseau.

14. Checklists et aide-mémoire

Révision fonctionnelle

- 1. Compte et rôle
- 2. Appartement actif
- 3. Disponibilité et vacances
- 4. Réservation en attente 24 h
- 5. Politiques acceptées
- 6. Paiement simulé
- 7. Confirmation + PDF
- 8. Tableau de bord

Révision technique

- 1. URL → vue → template
- 2. Formulaire valide les entrées
- 3. Service orchestre le métier
- 4. Modèle protège l'invariant
- 5. Transaction protège la concurrence
- 6. Test verrouille le comportement

7. Migration fait évoluer la base

8. Variables adaptent l'environnement

Test manuel après livraison

- Accueil, CSS, images et navigation s'affichent sur mobile et bureau.
- Inscription client et enseignant ; matricule obligatoire pour l'enseignant.
- Connexion locale, déconnexion POST et réinitialisation du mot de passe.
- Connexion Google sur le domaine de l'environnement.
- Recherche avec dates valides, passées, inversées et chevauchantes.
- Vacances : client bloqué, enseignant autorisé et réduction exacte.
- Réservation créée, numéro généré et e-mail de politiques reçu.
- Paiement interdit avant acceptation et après expiration.
- Paiement simulé valide : reçu, statut confirmé, e-mail et PDF lisible.
- Annulation avant paiement autorisée ; après paiement refusée.
- Administrateur : appartements, vacances, réservations, paiements, export Excel.
- Utilisateur normal : aucune donnée d'un autre client accessible par changement d'ID.

Fichiers à ouvrir selon la panne

Sujet	Fichiers principaux
Configuration globale	<code>webbap/settings.py</code> , <code>webbap/urls.py</code> , <code>.env.example</code>
Compte / permissions	<code>accounts/models.py</code> , <code>accounts/views.py</code> , <code>accounts/mixins.py</code>
Disponibilité / prix	<code>apartments/models.py</code> , <code>services/reservation_services.py</code>
Réservation / expiration	<code>reservations/models.py</code> , <code>reservations/views.py</code>
Paiement / reçu	<code>payments/models.py</code> , <code>payments/views.py</code> , <code>payments/forms.py</code>
E-mail / PDF	<code>services/confirmation_reservation.py</code> , <code>reservations/templates/reservations/emails/</code>
Notifications	<code>notifications/models.py</code> , <code>notifications/context_processors.py</code>
Indicateurs / export	<code>dashboard/views.py</code>
Présentation	<code>templates/base.html</code> , <code>static/css/site.css</code> , templates de l'application

Règle d'or. Avant de corriger une donnée, identifier l'environnement, sauvegarder, comprendre l'invariant métier, reproduire le problème, appliquer la correction la plus petite, puis exécuter les tests et le scénario utilisateur correspondant.

Fin du guide — généré à partir du code du projet FOSE Safar, sans inclure les valeurs secrètes du fichier `.env`.